

ATEC2026 线上赛选手指南

版本日期：2026 年 5 月 7 日

指南说明

- 1) 本指南为 ATEC2026 线上仿真挑战赛赛道 1 (Track 1: 运动优先) 与赛道 2 (Track 2: 操作优先) 全赛道统一适用的技术操作手册，涵盖任务说明、环境配置、接口规范、提交流程及评分规则。
- 2) 两个赛道共享相同的仿真环境和提交平台，区别仅在于 L0 阶段的入门任务不同。赛道 1 以任务 A (机器人徒步, Robot Hiking) 作为 L0 入门任务，侧重运动能力 (Legged Locomotion)；赛道 2 以任务 E (桌面整理, Table Clean-Up) 作为 L0 入门任务，侧重操作能力 (Manipulation)。L1 阶段全身运动与操作任务任务 B、任务 D, (Whole-Body Manipulation) 为两个赛道共享，最终排名以 L1 成绩为准。
- 3) 有关赛程时间安排、奖金体系、组队规则、知识产权与法务条款等信息，请参阅 ATEC2026 赛事官网及《参赛协议》。

题目介绍

欢迎来到 ATEC2026 仿真挑战赛。ATEC2026 仿真挑战赛分为两个递进阶段：L0 (初级阶段) 和 L1 (高级阶段)。参与者必须成功完成 (有过一次提交且得分大于等于 0.01 则视为成功完成) L0 中的任务才能获得参与 L1 任务的资格。所有任务必须由机器人完全自主完成。

L0 阶段 - 初级任务 (单点能力)

L0 阶段专注于基础机器人能力，包括移动和操作。

任务 A：机器人徒步

在此任务中，机器人需要穿越约 400 米的赛道，包含多样化的地形条件。地形类型包括：

- 平坦地面
- 不平整地形
- 倾斜表面（上坡和下坡）
- 多阶楼梯（上楼和下楼）

此任务主要评估机器人在不同环境条件下的移动稳定性和地形适应性。

任务 E：桌面整理

该任务评估机器人在结构化桌面环境中的基础操作能力。机器人需要：

- 从桌面表面抓取物体
- 处理 3 种不同类别的物体
- 将物体放入指定的粉色目标容器中

该任务主要评估机器人感知鲁棒性和抓取准确性。

L1 阶段 - 高级任务（系统集成）

L1 阶段引入更复杂的问题，需要实现机器人移动和操作能力的紧密结合。

任务 B：垃圾拾取与投放

在此任务中，机器人在 20x20 米的工作空间内操作，需要：

- 收集 18 个目标物体
- 处理多种物体类别
- 将物体放入指定区域

此任务评估机器人在杂乱环境中执行长程移动操作和自主规划的性能。

任务 D：越障

此任务专注于高级移动能力和环境交互。机器人需要：

- 穿越具有挑战性的结构，包括高架平台和沟壑
- 必要时改变环境（例如使用物体填充沟壑）
- 成功到达预定目标位置

此任务评估机器人的空间理解和长程推理能力。

机器人平台

本挑战赛提供多种标准化的机器人平台，以支持赛题所涉及的移动和操作任务。总共提供五款不同的机器人，涵盖广泛的运动和操作能力。

机器人平台：

- 机械臂（Manipulator）：轻质固定基 6 自由度机械臂，并配备 2 自由度夹爪，用于任务 E。
- 四足机械臂移动操作平台（Quadruped with Manipulator）：采用四足移动平台和轻质 6 自由度机械臂，用于任务 A、任务 B 和任务 D。
- 四轮足机械臂移动操作平台（Wheel-Legged Quadruped with Manipulator）：采用四轮足移动平台和 6 自由度机械臂，用于任务 A、任务 B 和任务 D。
- 二轮足机械臂移动操作平台（Wheeled-Biped with Manipulator）：采用二轮足移动平台和 6 自由度机械臂，用于任务 A、任务 B 和任务 D。
- 人型机器人（Humanoid Robot）：采用 29 自由度人形机器人并配备 2 自由度夹爪，用于任务 A、任务 B 和任务 D。

传感器配置：

所有机器人都配备标准化的传感器套件，以确保跨任务跨平台的一致性：

- RGB 摄像头：用于视觉感知和场景理解
- 深度摄像头：用于 3D 几何重建和距离估计
- 激光雷达 (LiDAR)：用于地形感知和环境建模
- 接触传感器 (Contact Sensor)：用于碰撞检测

评测指标

任务 A 评分

评估基于地形穿越和任务完成情况：

- 通过平坦地形：+2 分
- 通过崎岖地形：+4 分
- 通过斜坡：+8 分
- 通过楼梯：+8 分
- 到达终点：+4 分

最高总分：26 分。

任务 E 评分

任务 E 评估精确的桌面操作，包括抓取和放置。

- 每次成功抓取得+3 分
- 正确放置到目标区域得+3 分

最高得总分：18 分。

任务 B 评分

任务 B 评估机器人在大规模环境中执行物体收集和放置的能力。对于每个物体：

- 成功拾取物体：+1 分
- 正确放置物体到目标区域：+1 分

总共有 18 个物体，每个物体都计入总分。最高总分：36 分。

任务 D 评分

任务 D 评估机器人穿越障碍并与环境交互以完成任务的能力：

- 到达障碍物区域起点：+2 分
- 成功穿越障碍（平台或沟壑）：+20 分
- 利用箱子克服高架平台或跨越沟壑（环境改造）：+14 分

最高总分：36 分。

说明：

各任务分数不可直接比较，最终排名以 L1 综合成绩为准。

评分设计旨在平衡任务完成度、复杂性与系统稳定性。

本指南提供 ATEC2026 线上仿真挑战赛的完整技术说明，包括任务定义、环境配置、接口规范、评分规则及提交流程。

快速开始：

1. 安装 Isaac Lab 环境
2. 运行示例任务（scripts/view_task_*.py）
3. 修改 demo/solution.py
4. 本地验证
5. 提交评测

典型参赛流程概括 (Typical Pipeline)

参赛流程遵循从算法开发到在线评测的标准闭环，整体流程如下：

1. 开发与训练 (Develop & Train)

在本地或集群环境中进行策略训练。选手可基于官方提供的仿真环境与例程 baseline（如 RL / ACT），结合自定义方法（如强化学习、模仿学习或优化控制）进行系统开发、模型训练与优化。

2. 验证 (Validate)

在本地环境中对训练好的策略进行验证，使用官方提供的脚本（如 scripts/play_atec_task.py）进行可视化测试与调试，确保算法在不同任务和机器人平台上运行稳定，并满足时间与资源约束。

3. 打包 (Package)

将最终解决方案封装为标准提交格式。核心包括：

- 实现 demo/solution.py 作为统一入口
- 准备模型文件（如 policy.pt）与依赖（requirements.txt）
- 确保程序可在 **无网络环境** 下独立运行
- 控制初始化时间（需在 300 秒内完成启动）

4. 提交 (Submit)

通过平台提交解决方案：

- 方式一：上传源码，由平台自动构建镜像
- 方式二：自行构建 Docker 镜像并推送
- 提交后系统将自动触发评测流程。

5. 评测 (Evaluate)

评测系统将：

- 启动仿真环境与选手容器
- 通过 HTTP 接口与选手算法交互 (/step)
- 在规定时间内运行完整任务（最长 30 分钟墙钟时间）
- 根据任务完成情况自动计算最终得分并更新排行榜

6. 反馈结果与优化 (Feedback and Improve)

根据评分与榜单的反馈，选手持续优化算法与模型。

排行榜规则 (Leaderboard Logic)

ATEC2026 线上赛采用分阶段排行榜机制，以确保公平性并突出系统级能力评估。排行榜由 **L0 (初级阶段)** 与 **L1 (高级阶段)** 两部分组成，但两者作用不同：

1. L0 阶段 (入门与资格筛选)

- L0 阶段用于评估基础能力（运动或操作）。
- 每个任务 (Task A / Task E) 均设有独立排行榜。
- 选手只需在任一 L0 任务中获得**有效得分**，即可解锁 L1 阶段。
- **L0 成绩不计入最终排名，仅用于晋级资格判断。**

2. L1 阶段 (最终排名依据)

- L1 阶段评估系统级能力（移动 + 操作 + 环境交互）。
- 包含 Task B（垃圾拾取与投放）与 Task D（越障与环境交互）。
- 最终排行榜基于 **L1 综合表现** 进行排序。

L1 总分计算方式：

- $\text{Final Score} = \text{Score}(\text{Task B}) + \text{Score}(\text{Task D})$
- 排名依据总分从高到低排序。
- 若出现同分情况，将依次比较：
 1. Task B 得分（优先）
 2. Task D 得分
 3. 完成时间（更短者优先）
 4. 若 B 分、D 分、用时均相同，则最先达成该得分的队伍排名在前（以系统后台最终提交的时间戳为准）

3. 提交与排行榜更新

- 每次成功评测后，系统会自动更新排行榜。
- 每支队伍在排行榜中仅保留**历史最高成绩（Best Score）**。
- 排行榜为实时更新，选手可随时查看当前排名。

4. 晋级机制

- L1 排行榜将作为晋级线下赛（Real-World Preliminary）的主要依据。
- 具体晋级名额及规则请参考赛事官网说明。

核心原则：

排行榜强调系统级能力（L1），鼓励选手从单一能力（L0）走向完整任务闭环。

比赛环境介绍

本仓库基于 Isaac Lab v2.3.2 开发和测试。

安装与设置

第一步、安装 Isaac Lab v2.3.2

参考 Isaac Lab 官方文档自行安装，地址：https://isaac-sim.github.io/IsaacLab/v2.3.2/source/setup/installation/isaacsim_pip_installation.html。

第二步、下载代码

赛题代码包含两部分：ATEC2026_Simulation_Challenge 和 atec_robot_model。其中，ATEC2026_Simulation_Challenge 包含仿真环境代码和 Demo 代码，atec_robot_model 包含仿真环境用到的素材、机器人模型、ACT demo 的训练结果文件等非文本型的文件。可以通过 Github 和官网两个途径下载。

方式一、通过 Github 下载

```
# 通过 git 克隆代码
git clone https://github.com/atecup/ATEC2026_Simulation_Challenge

# 下载 atec_robot_model 到 ATEC2026_Simulation_Challenge 目录下
cd ATEC2026_Simulation_Challenge
curl https://static.atecup.com/atec2026/atec_robot_model.zip -o atec_robot_model.zip
unzip atec_robot_model.zip -d atec_robot_model
```

方式二、通过官网下载

在提交页面，点击「资源入口」标签页，然后点击「下载 Demo」按钮即可开始下载。下载后的文件名为 ATEC2026_Simulation_Challenge.zip，解压到当前目录即可。

注意：该目录下已经包含了 atec_robot_model 的内容，无需额外下载。

第三步、安装 ATEC 扩展：

```
cd ATEC2026_Simulation_Challenge/source/atec_rl_lab
pip install -e .
```

提示：该命令需要在 Isaac Lab 所在的环境下执行（跟你安装 Isaac Lab 的方式有关）。

安装完成后，所有 ATEC-* 任务将可在对应的虚拟 python 环境中使用。

竞赛环境列表

atec_rl_lab.tasks 模块将所有竞技场-机器人组合注册为 Gym 兼容环境，为评估和提交提供统一接口。

竞技场 / 机器人	G1	Tron1+Piper	B2+Piper	B2w+Piper	Piper
任务 A	ATEC-TaskA-G1	ATEC-TaskA-Tron1Piper	ATEC-TaskA-B2Piper	ATEC-TaskA-B2wPiper	
任务 B	ATEC-TaskB-G1	ATEC-TaskB-Tron1Piper	ATEC-TaskB-B2Piper	ATEC-TaskB-B2wPiper	
任务 D	ATEC-TaskD-G1	ATEC-TaskD-Tron1Piper	ATEC-TaskD-B2Piper	ATEC-TaskD-B2wPiper	
任务 E					ATEC-TaskE-Piper

注意：提供的环境专为评估和提交设计，不支持并行训练。

环境检查

```
cd ATEC2026_Simulation_Challenge
python scripts/list_envs.py

python scripts/view_robots.py --enable_cameras # 检查机器人模型
python scripts/view_task_a.py --enable_cameras # 任务 A 可视化
python scripts/view_task_e.py --enable_cameras # 任务 E 可视化
python scripts/view_task_b.py --enable_cameras # 任务 B 可视化
python scripts/view_task_d.py --enable_cameras # 任务 D 可视化
```

Demo 说明

目录结构

```
ATEC2026_Simulation_Challenge/demo/
├── Dockerfile          # 示例 Dockerfile
├── __init__.py         # 仅在本地调试时作为 python package 使用
├── requirements.txt    # (可选) 选手可在此文件中添加依赖
├── run.sh              # (必需) 在平台打分时的启动脚本, 不能更改
├── server.py           # (必需) HTTP 服务文件, 不能更改
├── solution.py         # (必需) 选手解决方案核心代码
├── solution_zero.py   # 全 0 demo solution
├── solution_rl.py      # RL demo solution
├── policy.pt           # RL demo 训练结果文件
├── solution_act.py     # ACT demo solution
├── act                 # ACT demo 的其它代码
└── policy_act.pt       # ACT demo 训练结果文件 (选手需要自行将
                        # ../atec_robot_model/baseline/act/policy.pt
                        # 拷贝为 policy_act.pt)
```

实现要求

总概

1. 修改 solution.py
2. 实现 AlgSolution.predict()
3. 返回 action

推荐流程:

1. 使用 baseline
2. 本地验证

3. 提交

参赛者必须实现 demo/solution.py，且文件名不能更改。该文件同时作为 scripts/play_atec_task.py 和 demo/server.py 的入口，前者用于本地验证，后者用于线上提交。demo/solution_zero.py、demo/solution_rl.py、demo/solution_act.py 三份文件为不同的解决方案文件，实际使用时需要将其更名为 demo/solution.py。

- 类名：AlgSolution
- 函数：predicts(obs, current_score)，其中 obs 为当前机器人状态，current_score 为当前所获得分
- 返回值：{"action": action, "giveup": False}，其中 action 为机器人当前控制量（需以 List 形式提供），giveup 为是否放弃标志（若为 True 则直接结束任务）

本地验证

```
python scripts/play_atec_task.py \  
    --task ATEC-TaskA-G1 \  
    --enable_cameras \  
    --debug
```

scripts/play_atec_task.py 在运行时会加载 demo/solution.py 作为唯一入口，选手如果要尝试更换解决方案，需要将解决方案文件更名为 demo/solution.py，或者在 scripts/play_atec_task.py 中更改 import 逻辑。demo/solution_rl.py、demo/solution_act.py 中读取的模型文件可以在 atec_robot_model 目录中找到，选手可以自行更换文件路径。

参数说明：

- --task：任务和机器人选择
- --debug：打印运行时间和得分

环境交互接口介绍

机器人状态反馈

任务提供对应的机器人状态反馈，类型为 dict 类型，可通过 key 索引到对应反馈数据，key 值如下

(1) 机器人本体状态反馈 (Proprio)

机器人本体状态反馈提供有关机器人状态的信息。定义如下：

- base_lin_vel: 机器人在本体坐标系中的基体绝对线速度
- base_ang_vel: 机器人在本体坐标系中的基体角速度
- velocity_commands: 基体期望速度，SE(2)（例如目标线速度和角速度）
- projected_gravity: 投影到机器人本体坐标系中的重力向量
- joint_pos: 相对关节位置（保持关节顺序）
- joint_vel: 相对关节速度（保持关节顺序）
- actions: 在上一个时间步控制量

访问方式：

```
obs["proprio"]
```

joint_pos、joint_vel 和 actions 的顺序在所有环境中都是固定且一致的。顺序遵循特定于机器人的关节配置：

各机器人关节顺序：

B2Piper(四足机械臂移动操作平台)：

```
FR_hip_joint, FR_thigh_joint, FR_calf_joint,  
FL_hip_joint, FL_thigh_joint, FL_calf_joint,  
RR_hip_joint, RR_thigh_joint, RR_calf_joint,  
RL_hip_joint, RL_thigh_joint, RL_calf_joint,  
arm_joint1, arm_joint2, arm_joint3, arm_joint4,  
arm_joint5, arm_joint6, arm_joint7, arm_joint8
```

B2wPiper(四轮足机械臂移动操作平台):

```
FR_hip_joint, FR_thigh_joint, FR_calf_joint,  
FL_hip_joint, FL_thigh_joint, FL_calf_joint,  
RR_hip_joint, RR_thigh_joint, RR_calf_joint,  
RL_hip_joint, RL_thigh_joint, RL_calf_joint,  
FR_foot_joint, FL_foot_joint, RR_foot_joint, RL_foot_joint,  
arm_joint1, arm_joint2, arm_joint3, arm_joint4,  
arm_joint5, arm_joint6, arm_joint7, arm_joint8
```

Tron1Piper (二轮足机械臂移动操作平台) :

```
abad_L_Joint, hip_L_Joint, knee_L_Joint,  
abad_R_Joint, hip_R_Joint, knee_R_Joint,  
wheel_L_Joint, wheel_R_Joint,  
arm_joint1, arm_joint2, arm_joint3, arm_joint4,  
arm_joint5, arm_joint6, arm_joint7, arm_joint8
```

G1 (人形机器人) :

```
left_hip_pitch_joint, left_hip_roll_joint, left_hip_yaw_joint,  
left_knee_joint, left_ankle_pitch_joint, left_ankle_roll_joint,  
right_hip_pitch_joint, right_hip_roll_joint, right_hip_yaw_joint,  
right_knee_joint, right_ankle_pitch_joint, right_ankle_roll_joint,  
waist_yaw_joint, waist_roll_joint, waist_pitch_joint,  
left_shoulder_pitch_joint, left_shoulder_roll_joint, left_shoulder_yaw_joint,  
left_elbow_joint, left_wrist_roll_joint, left_wrist_pitch_joint,  
left_wrist_yaw_joint,  
right_shoulder_pitch_joint, right_shoulder_roll_joint, right_shoulder_yaw_joint,  
right_elbow_joint, right_wrist_roll_joint, right_wrist_pitch_joint,  
right_wrist_yaw_joint,  
left_hand_Joint1_1, left_hand_Joint2_1,  
right_hand_Joint1_1, right_hand_Joint2_1
```

Piper (固定基机械臂) :

```
joint1, joint2, joint3, joint4,  
joint5, joint6, joint7, joint8
```

(2) 图像反馈 (Image)

图像反馈组提供来自机载摄像头的多视角视觉输入，支持导航和操作任务的感知：

- head_rgb: 来自头部安装摄像头的 RGB 图像（全局场景视角）
- head_depth: 来自头部安装摄像头的深度图像
- ee_rgb: 来自末端执行器摄像头的 RGB 图像（操作视角）
- ee_depth: 来自末端执行器摄像头的深度图像
- ee_dual_rgb: 来自双末端执行器摄像头的 RGB 图像（仅人形机器人平台可用）
- ee_dual_depth: 来自双末端执行器摄像头的深度图像（仅人形机器人平台可用）
- video_rgb: 来自桌面(眼在手外)摄像头的 RGB 图像 (仅 Task E 可用)
- video_depth: 来自桌面(眼在手外)摄像头的深度图像 (仅 Task E 可用)

访问方式：

```
obs["image"]
```

(3) 外部感知反馈 (Extero)

外部感知反馈组提供有关外部环境的信息，主要用于地形感知和导航：

- lidar_scan: 从机载激光雷达传感器获取的基于激光雷达的地形点云信息。

访问方式：

```
obs["extero"]
```

机器人控制

本次挑战赛中的机器人控制量按关节类型划分。腿部关节和机械臂关节使用关节位置命令控制，轮足机器人的轮子关节使用关节速度命令控制。控制配置如下：

```
joint_pos_leg = mdp.JointPositionActionCfg(  
    asset_name="robot",  
    joint_names=[""],  
    scale=0.5,  
    use_default_offset=True,  
    clip=None,  
    preserve_order=True,
```

```

)

joint_vel_wheel = mdp.JointVelocityActionCfg(
    asset_name="robot",
    joint_names=[""],
    scale=5.0,
    use_default_offset=True,
    clip=None,
    preserve_order=True,
)

joint_pos_arm = mdp.JointPositionActionCfg(
    asset_name="robot",
    joint_names=[""],
    scale=0.5,
    use_default_offset=True,
    clip=None,
    preserve_order=True,
)

```

其中，腿部和机械臂的位置控制量在应用到机器人前按 0.5 的因子缩放；轮子速度控制量按 5.0 的因子缩放。不同机器人会根据自身结构启用对应的 action 项：普通足式机器人（人形机器人，四足机械臂移动操作平台，机械臂）不启用轮子速度控制，轮足机器人（二轮足机械臂移动操作平台，四轮足移动操作平台）启用轮子速度控制。各机器人关节控制顺序和关节状态反馈顺序相同。

示例解决方案

本节为移动和操作任务提供示例解决方案，包含训练、评估和部署工作流程。

我们提供两个示例：

- 强化学习（RL）用于移动
- 模仿学习（ACT）用于操作

两者都遵循统一的工作流程：训练 → 评估 → 本地验证 → 线上提交

（1）强化学习示例

对于移动和移动操作任务（任务 A、任务 B、任务 D），提供了基于 PPO 的强化学习（RL）示例，实现在 `atec_rl_lab.train.locomotion` 中。

训练：

从仓库根目录运行以下命令：

```
python scripts/rs1_rl/train.py \  
  --task ATEC-Isaac-Velocity-Flat-Unitree-B2-v0 \  
  --headless \  
  --video
```

- `--headless`：关闭可视化
- `--video`：保存视频

评估：

训练后，使用以下命令评估策略：

```
python scripts/rs1_rl/play.py \  
  --task ATEC-Isaac-Velocity-Flat-Unitree-B2-v0
```

可以使用以下命令进行本地验证，注意需要将 `demo/solution_rl.py` 更名为 `demo/solution.py` 使用

```
cd ATEC2026_Simulation_Challenge  
python scripts/play_atec_task.py \  
  --task ATEC-TaskA-B2Piper \  
  --enable_cameras \  
  --debug
```

（2）模仿学习示例

针对操作任务（任务 E），提供了基于 ACT（Action Chunking with Transformers）的模仿学习示例，实现在 `atec_rl_lab.train.act` 中。

收集演示数据：

```
python scripts/act/collect_demos_task_e.py \  
    --pick_objects 3 --num_demos 100 \  
    --headless --enable_cameras --save_images
```

筛选数据集：

```
python scripts/act/filter_demos.py \  
    --input datasets/atec_task_e/trajectory.hdf5 \  
    --output datasets/atec_task_e/trajectory_filtered.hdf5 \  
    --threshold 0.001
```

训练策略：

```
cd scripts/act  
bash baseline.sh
```

可以使用以下命令进行本地验证，注意需要将 demo/solution_act.py 更名为 demo/solution.py 使用

```
cd ATEC2026_Simulation_Challenge  
python scripts/play_atec_task.py \  
    --task ATEC-TaskE-Piper \  
    --enable_cameras \  
    --debug
```

线上打分

线上打分时，选手的解决方案与仿真环境运行在两个容器中，由打分程序负责传递信息。选手的解决方案被包装为一个 HTTP 服务器（参考 sever.py），打分程序通过 HTTP 协议与解决方案容器交互，通过 Gym API 与仿真环境交互。

选手的解决方案容器启动后，平台会通过探测 /health 接口来判断 HTTP 服务是否启动成功，超时时间 300 秒，即选手需要在 300 秒内让自己的代码完成初始化，让 server 正常启动。反之，超过 300 秒 /health 返回失败，会被视为服务启动失败。

打分程序首先通过 Gym 的 `env.reset()` 初始化仿真环境，并获取初始仿真环境信息和机器人观测信息，并调用 `/step` 接口，将这些信息传递给选手的解决方案，解决方案里需要根据这些信息预测机器人的下一步动作指令，并同步返回。打分程序会将动作指令通过 Gym 的 `env.step()` 传递给仿真环境，并获取新的仿真环境信息和机器人观测信息，然后调用 `/step` 接口传递给解决方案，如此循环执行。

打分程序从 `env.reset()` 后开始计时，超过 30 分钟后，会强制终止执行，并返回当前得分为选手的最终分数。

选手解决方案终止后，打分程序会调用 HTTP 协议的 `/quit` 接口通知选手容器退出。

提交方式

总概

上传方式：

1. 源码上传
2. 镜像上传

每日提交限制（UTC+8 10:00 重置）：

- 提交次数：最多 10 次（包含失败）
- 出分次数：最多 3 次（仅成功评测）

规则：

- 构建失败 → 不计入提交
- 启动失败 → 计入提交
- 成功出分 → 计入两者

方式一、源码上传

本赛事支持源码上传的方式，您只需要提交源码，系统自动完成镜像构建和打分。

操作流程：

1. 进入提交页面，选择「上传源码」标签页；
2. 选择机器人构型；
3. 上传文件：
 - 3.1. 点击「上传文件」或「上传文件夹」按钮，选择本地代码和模型文件
 - 3.2. 点击「提交变更」按钮，开始上传文件
4. 确认目录结构，根目录下结构如下（可参考 Demo 的目录结构）：

```
├─ solution.py      # （必需）选手解决方案核心代码
├─ policy.pt        # （可选）训练结果文件
├─ requirements.txt # （可选）选手可在此文件中添加依赖
└─ 其它文件         # （可选）其它文件
```

5. 点击「提交打分」按钮，会执行构建镜像操作，构建成功后触发评分。

说明：

- 基础镜像环境为 Python 3.12、PyTorch 2.7.1、CUDA 12.8.1，操作系统为 Alinux 3 (2104)；
- 构建使用的 Dockerfile 参考 demo 目录下的 Dockerfile；

注意事项：

1. 请勿上传 run.sh 和 server.py 两个文件，构建镜像时系统会放入这两个文件，文件内容和 Demo 里的一致；
2. 请参考《附录：环境中已有的依赖清单》，避免依赖冲突；
3. 文件数量不能超过 300 个。

方式二、推送镜像

本赛事支持推送镜像的提交方式，您在本地构建好 docker 镜像，推送到指定仓库后，系统自动开始打分。

操作流程：

1. 进入提交页面，选择「推送镜像」标签页；
2. 选择网络接入点，选择机器人构型；
3. 点击「生成临时凭证」按钮，会生成三条 Docker 命令（登录、构建、推送）；
4. 在您的代码目录下依次执行三条命令，即可将镜像推送到 ATEC 服务器；

注意事项：

1. 您的目录结构须与 Demo 的目录结构保持一致，如果不一致的话，需要自行调整 docker 命令；
2. 镜像大小限制为 30G。超过 30G 时，提交记录状态为“前置检查失败”，点击右侧的感叹号图标，可以查看原因。该状态不扣减提交次数。

三条 docker 命令说明：

```
# 第一条命令：登录 docker 镜像仓库
docker login --username=cr_temp_user *****
```

```
# 第二条命令：构建 docker 镜像
docker build -t *****

# 第三条命令：推送到镜像仓库
docker push *****
```

提交限制

- 本赛事对提交频率实行双额度控制机制，以 24 小时为一个计时周期（T+0 日 10:00 至 T+1 日 10:00，UTC+8）。
 - 第一项额度为"提交次数"，每 24 小时上限 10 次。选手通过源码上传的方式成功构建镜像，或推送镜像的方式推送镜像成功后，会立即触发打分任务，此时会消耗一次提交次数。
 - 第二项额度为"出分机会"，每 24 小时上限 3 次。仅当提交成功进入评测流程并产出最终分数时，才消耗一次出分机会。
- 额度扣减规则如下：
 - 源码上传方式中镜像构建失败、推送镜像方式中生成临时凭证但未推送镜像等未触发服务启动的情况，不扣减提交次数。
 - 服务启动报错、/health 接口探测 300 秒无响应、/step 接口调用失败等进入打分阶段但未成功出分的情况，仅扣减提交次数，不扣减出分机会。
 - 任务成功出分（含因墙钟超时记零分的情况），同时扣减提交次数和出分机会。
 - 任一额度耗尽，均无法继续提交，需等待下一个 24 小时周期刷新。
- 其他限制：每个榜单每个赛队同时运行的任务数最大为 1。后台评测系统将自动根据赛队当前所处的级别进行评测。若赛队在当前级别的分数超过该级别的晋级分数线，则自动晋级至下一级别，同时提交次数和出分机会将重置。

提交记录

- 源码上传的提交方式中，构建镜像步骤成功后，产生提交记录，否则不产生提交记录；
- 推送镜像的提交方式中，推送镜像成功后，产生提交记录，否则不产生提交记录。

提交成功后，可以在“提交入口”标签页下方的提交记录中查看自己提交的镜像、机器人构型以及得分情况，可以在“历史提交”档签页，查看本赛队所有成员的提交记录。

计算资源

- 用于运行选手赛题镜像的 GPU 服务器配置如下：CPU 16 核，内存 24 GB，磁盘 500 GB，GPU 单卡 RTX 5880（总显存 48 GB）。
- 可用显存：约 34GB（仿真占用 ~14GB）。显存说明：GPU 显存由选手容器与同机仿真环境容器共享。仿真环境预估占用约 14 GB，选手实际可用显存约 34 GB。请在设计方案时合理规划模型大小，避免因显存不足（OOM）导致任务失败。
- 运行时间限制以物理真实世界（Wall Clock）的 30 分钟为准，即从任务开始执行起计算的真实物理时间(不包含镜像构建时间，任务调度时间)；超时后，根据任务完成情况，获取成功步骤得分，且占用当日出分机会。
- 网络限制：选手容器无法访问互联网。所有依赖和模型文件须在镜像构建阶段完成打包。

附录：技术建议（非评分项）

任务 A：

基于强化学习的示例方法已能够实现基础的平坦地形行走。后续可在多样化地形和域随机化条件下训练以提升机器人在其他地形上的表现。

任务 E：

基于 ACT 的示例方法已能够实现单一物体的抓取与放置，后续可收集多样化数据来控制策略的泛化性和精度。

任务 B (移动 - 操作) :

题目需要实现协同移动和操作。推荐方法:

- 基于端到端 强化学习方法: 如 deep whole body control
- 混合方法: 基于强化学习的控制策略用于基体控制, 基于传统控制方法如 IK / MPC 用于机械臂控制
- 最优控制: 基于全身模型预测控制协同控制腿足与机械臂

任务 D (基于视觉的穿越) :

题目需要机器人具有一定的空间理解与推理能力。建议:

- 在控制策略的基础上融入图像和深度信息实现端到端的控制策略, 如: extreme parkour

进阶方向:

- 视觉-语言-动作 (VLA)
- 视觉-语言模型 (VLM)

附录: 环境中已有的依赖清单

```
accelerate==1.2.1
accelerating-doc==0.0.4
annotated-types==0.7.0
anyio==4.13.0
asttokens==3.0.1
certifi==2024.12.14
charset-normalizer==3.3.2
click==8.3.3
decorator==5.2.1
deepspeed==0.16.9+ali
einops==0.8.0
executing==2.2.1
fastapi==0.136.0
filelock==3.15.4
fsspec==2024.6.0
h11==0.16.0
```



```
hjson==3.0.2
huggingface_hub==0.30.2
idna==3.7
ipython==9.12.0
ipython_pygments_lexers==1.1.1
jedi==0.19.2
Jinja2==3.1.4
MarkupSafe==2.1.5
matplotlib-inline==0.2.1
mpmath==1.3.0
msgpack==1.1.0
networkx==3.3
ninja==1.11.1.1
numpy==2.1.3
packaging==24.0
parso==0.8.6
pexpect==4.9.0
pillow==10.3.0
pip==24.2
prompt_toolkit==3.0.52
psutil==5.9.2
ptyprocess==0.7.0
pure_eval==0.2.3
py-cpuinfo==9.0.0
pydantic==2.10.4
pydantic_core==2.27.2
python-multipart==0.0.26
Pygments==2.20.0
PySocks==1.7.1
PyYAML==6.0.1
regex==2024.5.15
requests==2.32.4
safetensors==0.4.3
setuptools==80.9.0
stack-data==0.6.3
starlette==1.0.0
sympy==1.13.3
tokenizers==0.21.0
torch==2.7.1.8+cu128
torchaudio==2.7.1.8+cu128
torchvision==0.22.1.8+cu128
tqdm==4.66.4
traitlets==5.14.3
transformers==4.53.1
triton==3.2.0
typing_extensions==4.12.2
typing-inspection==0.4.2
urllib3==2.5.0
uvicorn==0.45.0
wcwidth==0.6.0
wheel==0.45.0
```

常见错误原因 (Common Failure Modes)

在提交与评测过程中，以下问题是导致任务失败或得分异常的常见原因。建议选手在提交前重点检查：

1. 服务启动失败 (/health 超时)

- 现象：提交后未进入评测，或显示服务启动失败
- 原因：
 - 初始化时间过长 (超过 300 秒)
 - 模型加载或依赖安装阻塞
- 建议：
 - 精简初始化流程 (避免在启动阶段进行复杂计算)
 - 提前加载模型并优化加载时间
 - 确保 /health 接口快速响应

2. 依赖冲突或环境不兼容

- 现象：程序在运行时崩溃或无输出
- 原因：
 - 与平台预装库 (如 PyTorch、CUDA) 版本冲突
 - 使用未打包的外部依赖 (平台无网络)
- 建议：
 - 避免覆盖基础环境中的核心依赖
 - 所有依赖必须在镜像构建阶段完成安装
 - 本地尽量对齐官方环境配置进行测试

3. 显存不足 (OOM)

- 现象：程序运行中断或无响应
- 原因：
 - 模型过大或推理开销过高
 - 未合理控制 batch size 或中间缓存
- 建议：
 - 控制模型规模与推理开销
 - 避免不必要的中间变量存储
 - 预留足够显存（仿真环境已占用部分资源）

4. 动作输出异常 (Invalid Action)

- 现象：机器人行为异常、得分极低或任务失败
- 原因：
 - 输出维度不匹配
 - 控制值范围不合理（过大或不稳定）
- 建议：
 - 严格检查 action 维度与关节顺序
 - 对输出进行裁剪或归一化
 - 避免高频震荡控制信号

5. 推理速度过慢 (Step 超时)

- 现象：任务提前终止或未完成

- 原因：
 - 单步推理时间过长
 - 使用复杂模型（如大规模 VLM / Transformer）
- 建议：
 - 优化推理速度（模型压缩、减少计算）
 - 避免在 /step 中执行重计算逻辑
 - 控制整体运行时间在限制范围内

6. 任务逻辑不完整 (Early Termination)

- 现象：任务提前结束或得分异常低
- 原因：
 - 错误触发 giveup=True
 - 未处理关键任务阶段（如未完成放置或越障）
- 建议：
 - 确保任务流程完整执行
 - 谨慎使用提前终止逻辑

7. 仿真与本地结果不一致

- 现象：本地运行正常，线上失败或得分显著下降
- 原因：
 - 本地环境与平台环境不一致
 - 未考虑随机性或边界情况

- 建议：
 - 使用官方环境进行本地验证
 - 增强策略鲁棒性（避免过拟合特定情况）
 - 测试多种初始条件与随机扰动

建议：在提交前，建议至少完成一次完整本地验证，并检查日志输出与资源使用情况，以确保解决方案在评测环境中稳定运行。

平台保障与支持

- 平台故障补偿：若因不可抗力或 ATEC 评测平台自身故障（包括但不限于服务器宕机、仿真环境异常中断、评分系统错误）导致选手任务非正常终止或评测结果异常，选手可通过下方支持渠道提交反馈。经平台核实确认后，将返还对应的提交次数和/或出分机会。因选手自身代码错误、依赖冲突、资源超限等原因导致的任务失败，不在补偿范围内。
- 技术支持渠道：如在环境配置、镜像提交、评测结果等方面遇到问题，请统一通过 ATEC2026 赛事官网首页的咨询中心入口进行反馈。如涉及规则争议或算力券问题，亦可发送邮件至 ATECservice@atecup.com。提交反馈时，请附上赛队名称及队长账号、对应的提交记录编号、以及问题的具体表现及截图（如有），以便快速定位问题。